SOLID Continue.

LSP Guidelines

Recap :→

If client expects A but if we give B (child class of A) & nothing fails, then LSP is followed !!

To follow, LSP, subclass | child should respond / behave like Parent class.

Guidelines

1. Signature Rule
2. Property Rule
3. Method Rule

---

① Signature Rule

Points to Remember

- ↳ Broad :→ Parent or Ancestor
- ↳ Narrow :→ Child or Descendent

Animal Class is Broader class of Dog and Dog Class is narrow class of Animal.

ⓐ Signature Rule :→ Method Argument Rule

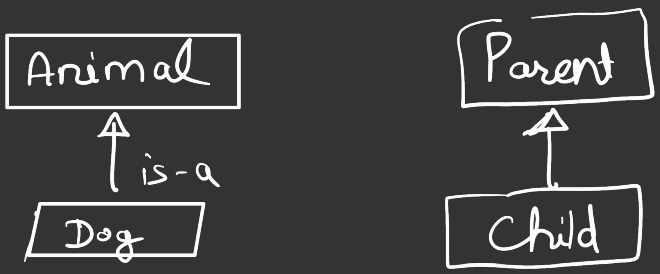
```
class Parent {
    solve (String s) {
    }
}
```

```
class Child extends Parent {
    solve (Type s) {
    }
}
```

This "Type" should be Broader class of String or String itself.

This rule is imposed by C++ & Java internally

(6) Signature Rule : Return Type Rule



→ class Parent {

```
    Animal random() {
        ≡≡≡
    }
}
```

class Child extends Parent {

```
    Type random() {
        ≡≡≡
    }
}
```

Can be Animal or its narrow types.

If "Type" is, narrow class then this is called "Covariance"

ⓒ Signature Rule :→ Exception Rule.

→

```
class Parent {
    m1() throws RuntimeExp {
    }
}
```

```
class child extends Parent {
    m1() throws Exp {
    }
}
```

this should be either RuntimeExp or its narrow exception.

client

```
try {
    P.m1();
}
catch (RuntimeExp e){
    ===
}
```

② Property Rule

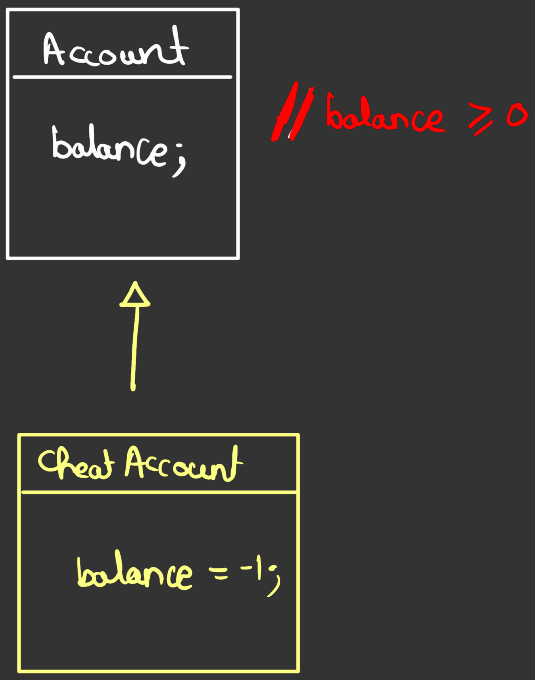
- Class Invariant
- History Constraint

ⓐ Class Invariant

↳ **Invariant** :→ Koi bhi aisa fact/Rule jo kisi class ke liye humesha "True" hoga.

Def^n :→ Let's say we have a class Parent aur iss parent class ke liye ek in-variant hai R_1. So Child has responsibility to follow this invariant R_1. Ya to as it is follow kare ya to isko strengthen kare but bilkul bhi weak na kare.

Eg :-> Suppose we have Account class & it has a variable balance. Rule is that balance is always greater than 0.



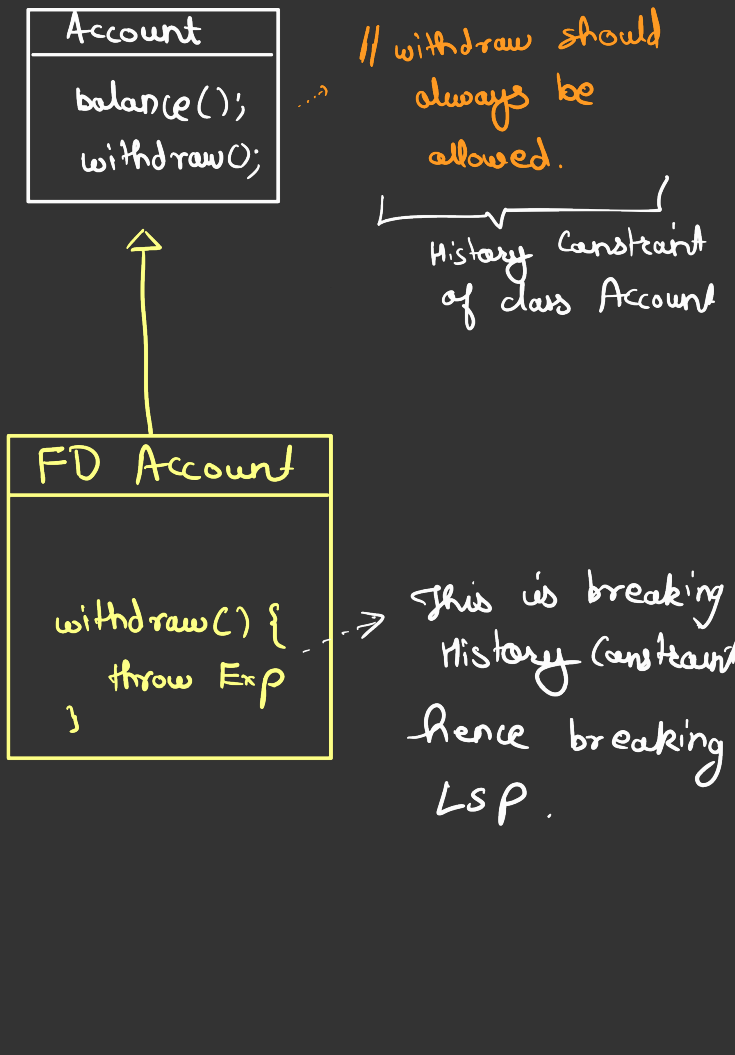
This cheatAccount class doesn't follow class Invariant Rule.

(6) History Constraint

→ It says "History kabhi change nahi honi chaiye".

Ek bar jab Parent class ne keh diya ki ye state hai then child class usi state ko follow kare.

Eg



Note :→

Immutable Class → Jisko koi inherit na kar paye

↳ final Keyword

Immutable Methods → Jisko koi Override na kar paye

↳ final Keyword.

Agar Parent class ne kuch chize immutable kar rakhi hai but child class unko mutable bana de to this is Violation of History Constraint.

③ Method Rule

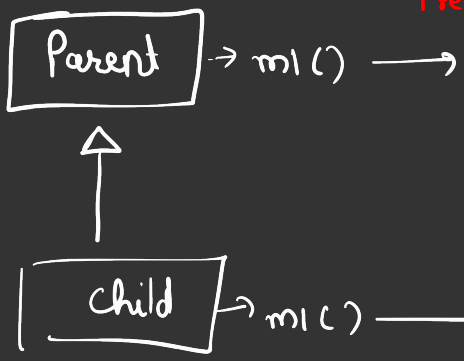
- Pre-Condition
- Post-Condition

ⓐ Pre-Condition

→ Condition which should be followed before execution of any method.

Let's say we have a parent class "P" which has a method m1() with a pre-condition. If child class "C" will either follow the pre-condition or weaken the pre-condition but should no strengthen it.

eg



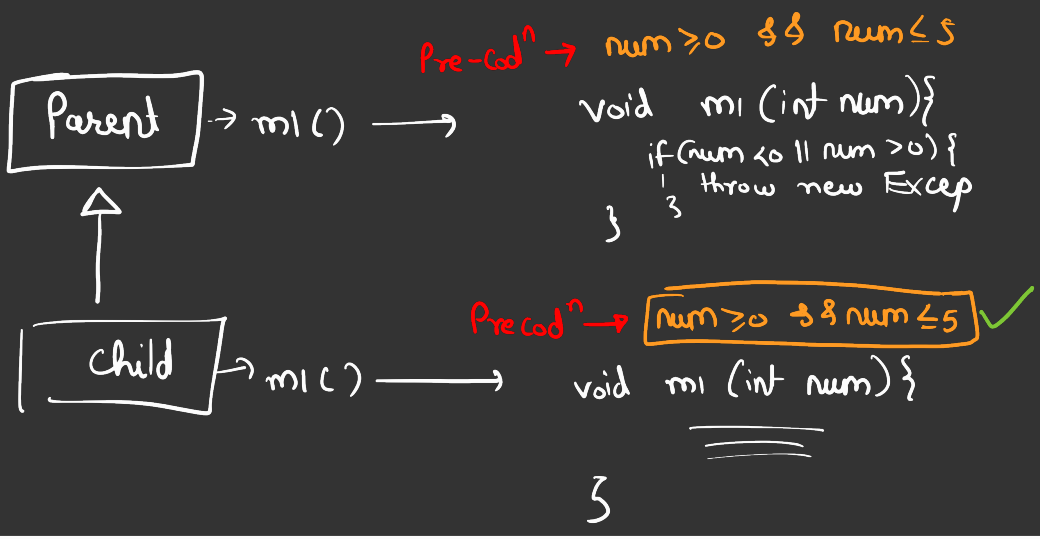
Pre-Cond^n → num ≥ 0 && num ≤ 5

```
void m1(int num){
    if(num < 0 || num > 0){
        throw new Excep
    }
}
```

Pre Cond^n → num ≥ 0 && num ≤ 10

```
void m1(int num){
```

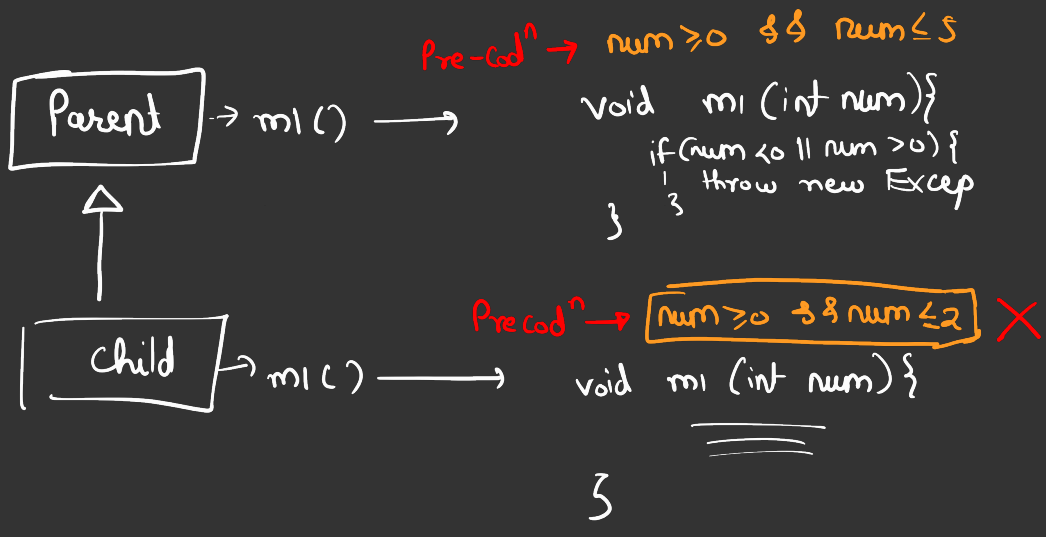
eg


Pre-cond^n → num ≥ 0 && num ≤ 5
Parent
m1()
void m1(int num){
if(num < 0 || num > 0){
throw new Excep
}
Pre cond^n → num ≥ 0 && num ≤ 5
Child
m1()
void m1(int num){

}

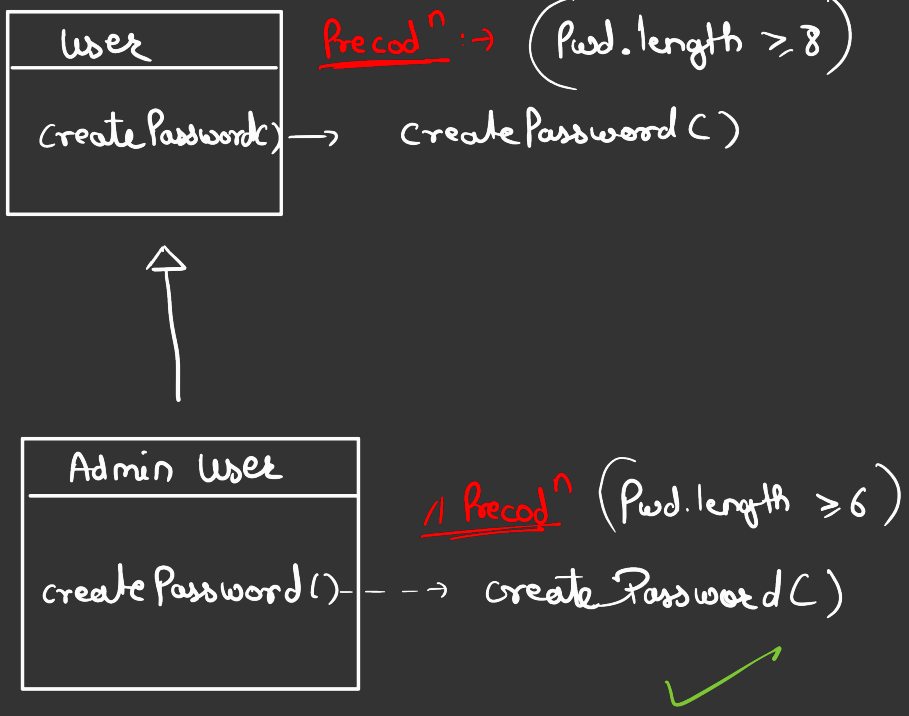
---

eg


Pre-cond^n → num ≥ 0 && num ≤ 5
Parent
m1()
void m1(int num){
if(num < 0 || num > 0){
throw new Excep
}
Pre cond^n → num ≥ 0 && num ≤ 2
Child
m1()
void m1(int num){

}

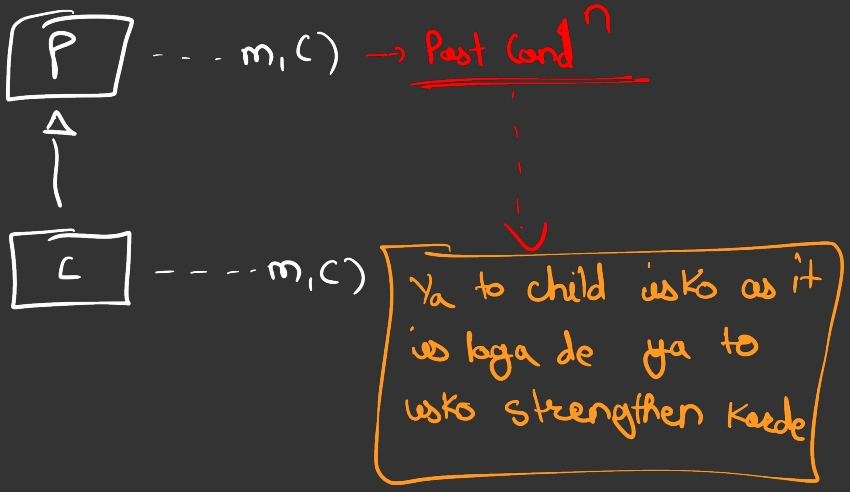
Practical Eg :→



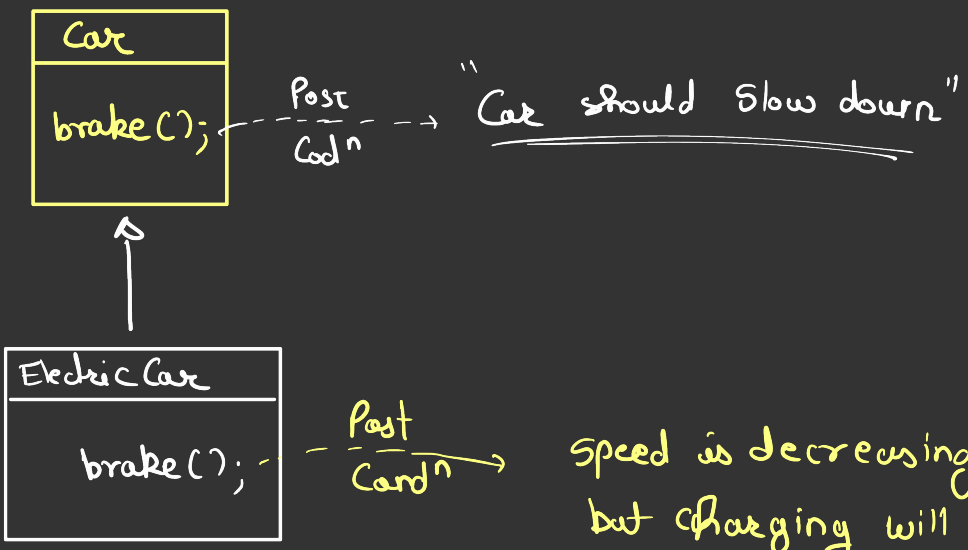
Although in this, Child should strictly follow Precond^n of Parent. Depends on use-case to use-case.

⑤ Post Condition

→ Post condition is condition which should be true after method execution.



Eg



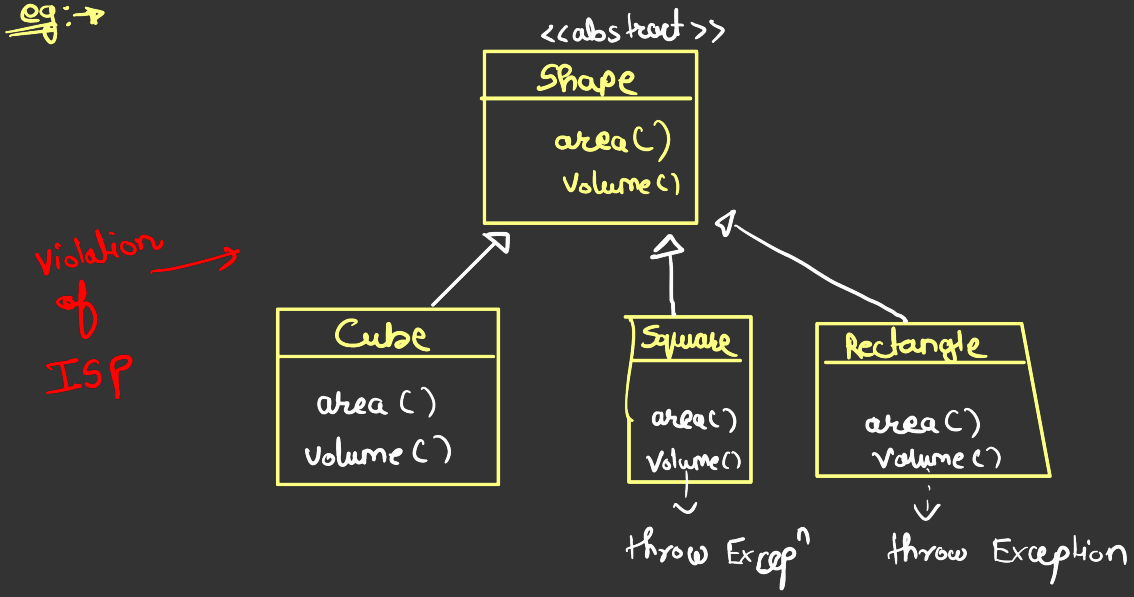
4 Interface Segregation Principle

↳ Many client specific interfaces are better than one general purpose interface.

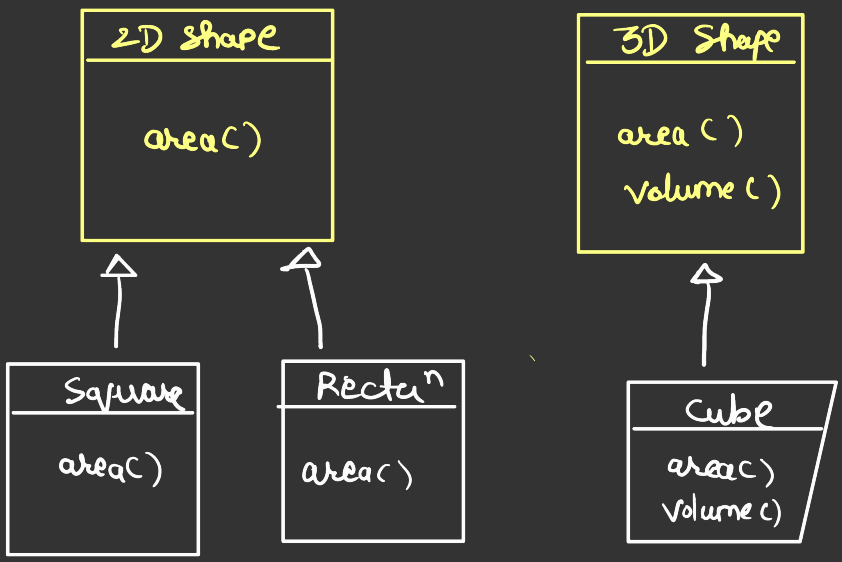
↳ Client should not be forced to implement methods they don't need.

---

eg:→



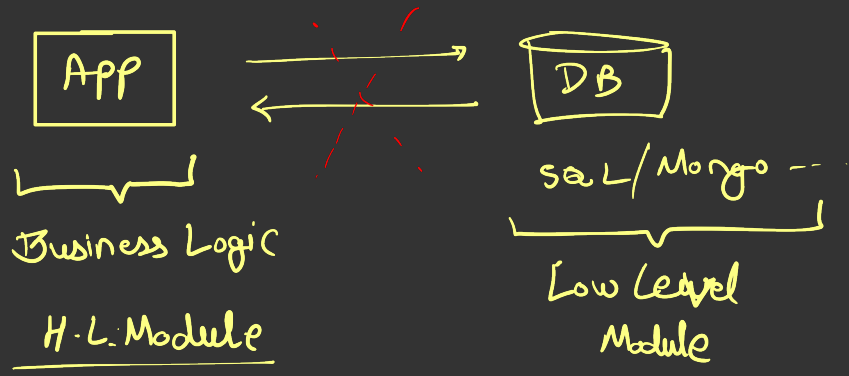
Correct way :->


2D Shape
area()
3D Shape
area()
volume()
Square
area()
Rectan
area()
Cube
area()
volume()

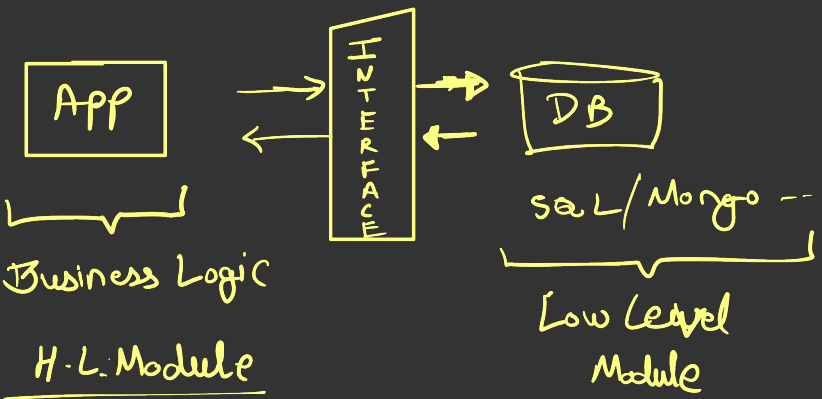
(5) Dependency Inversion Principle

↳ High level module should not depend on low level module but rather both should depend on abstraction.

Violation :-)

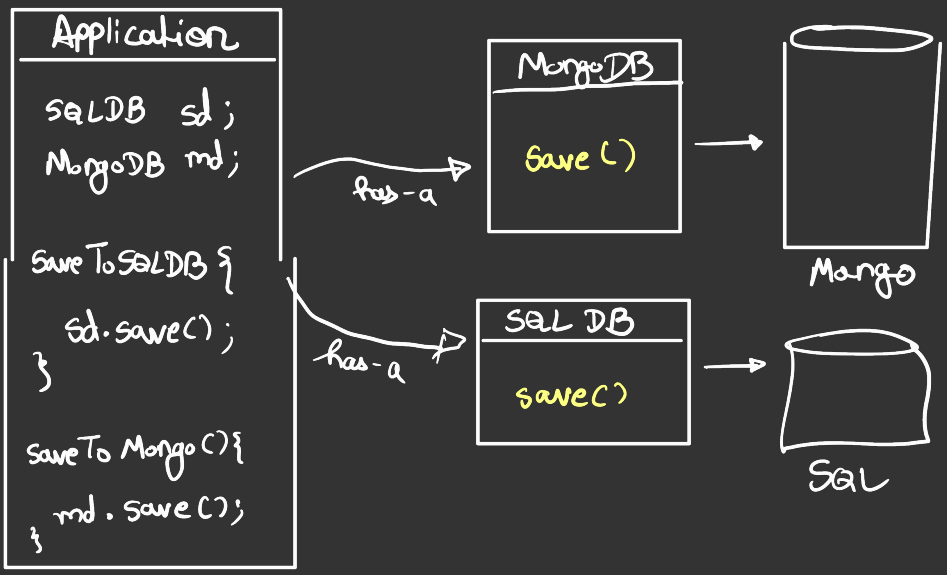


Correct way

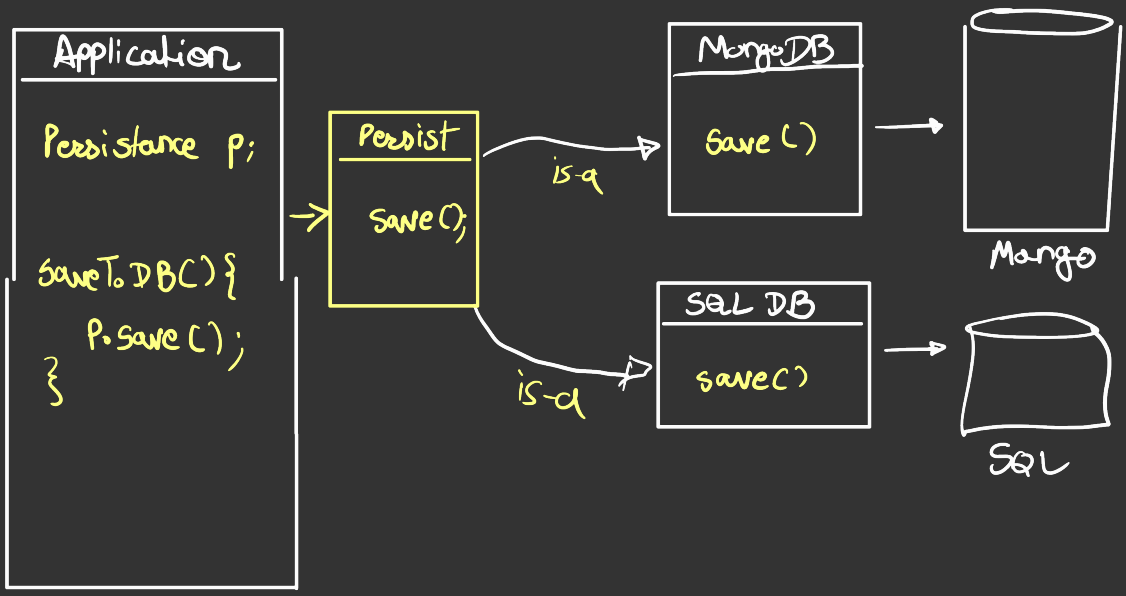


eg:->

Violation



eg:-)



Note :- If open-close Principal is the target then dependency injection is the solution.

Real life example of DIP